# 32 bit Modular Arithmetic and SIMD Vectorisation

Duc Vinh Nguyen, Henry Zheng

March 2026

## 1 Introduction

This project explores and optimize modular reductions for prime numbers $p < 2^{32}$ using SIMD Vectorisation (AVX2, AVX512, NEON) along with various different techniques and algorithmes.

**Objective:** The aim of this study is to be able to realize an optimized implementation with minimal performance cost for modular reductions.

## 2 Fundamental Cost of Modular Reduction

The most naive way to compute $a \pmod{n}$ is to perform the operation :

$$a - \left\lfloor \frac{a}{n} \right\rfloor \cdot n \tag{1}$$

However, on modern processor architectures (Intel, AMD,...), integer division is non-pipelined and has significantly higher computational cost than other operations like additions and multiplications.

Since equation (1) requires a divsion and substraction operation, its considered highly inefficient from both a software and hardware perspective.

## 3 Shoup's Modular Multiplication

Shoup's algorithm is centraled around precomputation as a work around the costly division operations

**Algorithm 1:** Shoup's modular multiplication algorithm

**Input** : $A, B \in [0, p[$, we suppose that $p < \beta/2$
precompute $B' = \lfloor B\beta/p \rfloor$
**Output:** $R = AB \bmod p$

$Q \leftarrow \lfloor AB'/\beta \rfloor$ ;
$R \leftarrow (AB - Qp) \bmod \beta$ ;
**if** $R \geq p$ **then**
  $R \leftarrow R - p$;

## 3.1 Input constraint

To ensure the correctness of the modular reduction and prevent register overflow during SIMD operations, the following conditions must be satisfied:

- **Word Size ($\beta$):** Set to $2^{32}$ for 32-bit modular arithmetic
- **Input 1 ($A$):** Integer defined over 32 bits
- **Input 2 ($B$):** Integers within the range $[0, p[$
- **Modulus ($p$):** A prime number defined over 31 bits ($p < 2^{31}$)

## 3.2 Theorem:

Let $p$ be a prime number such that $p < \beta/2$ where $\beta = 2^{32}$. Given integers $A$ an integer defined over 32 bits and $B \in [0, p)$, let $B'$ be the precomputed value $B' = \lfloor B\beta/p \rfloor$. If the estimated quotient is defined as $Q = \lfloor AB'/\beta \rfloor$, then the intermediate remainder $R = AB - Qp$ satisfies the following condition:

$$0 \leq AB - Qp < 2p$$

## 3.3 Procomputational Logic

The precomputed value $B'$ is defined over 32 bits as:

$$B' = \lfloor (B \cdot 2^{32})/p \rfloor$$

When computing the $a \cdot b \pmod p$ :

$$A \cdot B \pmod p = A \cdot B - \left\lfloor \frac{A \cdot B}{p} \right\rfloor \cdot p$$

We can rewrite this:

$$\begin{aligned} A \cdot B \pmod p &= A \cdot B - \left\lfloor \frac{A \cdot \left\lfloor \left(\frac{B \cdot 2^{32}}{p}\right) \right\rfloor}{2^{32}} \right\rfloor \cdot p \\ &\approx A \cdot B - \left\lfloor \frac{A \cdot B'}{2^{32}} \right\rfloor \cdot p \end{aligned}$$

### 3.4 Avantages:

Using the precomputed value B' to approxiamtely calculate $\left\lfloor \frac{A \cdot B}{p} \right\rfloor$ improves on the naive method by 2 ways:

1. **Storing the Inverse of $B$:** $B'$ is computed with 1 division and can be stored to be used in other modular reduction provided the same B provided the same $p$
2. **Shoup's algorithm replaces division operations (slow) with multiplications and additions (fast):** $\frac{A \cdot B'}{2^{32}}$ has only 1 multiplication operation and a bit shift to the right of 32 bits. On a CPU, both multiplication and bit shift operations can be done efficiently compared to division

### 3.5 Fundamental Principal

Given x the quotient of 2 32 bits integers:
The proof of this algorithm based on the non decimal value of ($\lfloor x \rfloor$):

$$x - 1 < \lfloor x \rfloor \leq x$$

We can rewrite as:

$$0 \leq x - \lfloor x \rfloor < 1$$

### 3.6 Basic Reduction B = 1:

### 3.7 Demonstration:

The algorithm uses the precomputation $B' = \lfloor B\beta/p \rfloor$ such that:

$$0 \leq \frac{B\beta}{p} - \left\lfloor \frac{B\beta}{p} \right\rfloor < 1$$

$$0 \leq \frac{B\beta}{p} - B' < 1 \tag{2}$$

Using the same logic, the algorithm estimates the quotient $Q = \lfloor AB'/\beta \rfloor$ such that:

$$0 \leq \frac{AB'}{\beta} - \left\lfloor \frac{AB'}{\beta} \right\rfloor < 1$$

$$0 \leq \frac{AB'}{\beta} - Q < 1 \tag{3}$$

We now prouve that remainder $R = (A \cdot B - Q \cdot p)$ is within the interval [0, 2p[:

We firstly multiply the equation (1) by $Ap/\beta$:

$$0 \leq \left( \frac{B\beta}{p} - B' \right) \cdot \frac{Ap}{\beta} < 1 \cdot \frac{Ap}{\beta}$$

$$0 \leq AB - \frac{B'Tp}{\beta} < \frac{Ap}{\beta} \tag{4}$$

Similarly, we multiply the equation (2) by $p$:

$$0 \leq \left(\frac{AB'}{\beta} - Q\right) \cdot p < 1 \cdot p$$

$$0 \leq \frac{AB'p}{\beta} - Qp < p \tag{5}$$

By adding the equation (3) and (4), we deduce:

$$0 \leq (AB - \cancel{\frac{AB'p}{\beta}}) + (\cancel{\frac{AB'p}{\beta}} - Qp) < \frac{Ap}{\beta} + p$$

Simplifying this result, we are left with:

$$0 \leq AB - Qp < p + \frac{Ap}{\beta}$$
$$0 \leq AB - Qp < p \cdot (1 + \frac{A}{\beta})$$

By constraint, $A \in [0, p[$ and $p < \beta$ thus $A < \beta$. Therefore we can rewrite:

$$0 \leq AB - Qp < p \cdot (1 + \frac{A}{\beta}) < p \cdot (1 + \frac{\beta}{\beta})$$
$$0 \leq AB - Qp < p \cdot (1 + 1)$$
$$0 \leq AB - Qp < 2p$$
$$0 \leq R < 2p$$

We have now prouven that $R \in [0, 2p[$ the final if statement of the helps correct the remainder. If $R > p$ the modular reduction needs 1 more step $R - p$, if not $R$ is the correct result

## 3.8 SIMD Vectorization Implementation

We decided to create 2 versions for implementating a high performance Shoup algorithm

### 3.8.1 AVX

For the AVX version, we decided to use 2 main AVX function: _mm256_mul_epu32 and _mm256_mul_epi32.

#### Intel _mm256_mul_epu32

Listing 1: Auxilary Function of Shoup's Modular Reduction

```
    static inline __m256i _shoup_avx(__m256i va,
                                     __m256i vb,
                                     __m256i vb_precomp,
                                     __m256i vp,
                                     __m256i mask32)
{
    // 1. q = (a * b_precomp) >> 32:
    __m256i q = _mm256_mul_epu32(va, vb_precomp);
    q = _mm256_srli_epi64(q, 32);

    // 2. ab:
    __m256i ab = _mm256_mul_epu32(va, vb);

    // 3. qp:
    __m256i qp = _mm256_mul_epu32(q, vp);

    // 4. c = (ab - qp) & mask:
    __m256i c = _mm256_sub_epi64(ab, qp);
    c = _mm256_and_si256(c, mask32);

    // 5. if (c >= p) c -= p
    __m256i compare = _mm256_cmpgt_epi64(vp, c);
    __m256i sub_mask = _mm256_andnot_si256(compare, vp);

    return _mm256_sub_epi64(c, sub_mask);
}
```

Listing 2: AVX2 Implementation of Shoup's Modular Reduction

```
    static Vector _shoup_scale_avx(Parameters param, Vector v)
{
    /*1. Preloading b, b_precomp, p and 32 bit mask*/
    int size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);
    __m256i mask_32 = _mm256_set1_epi64x(0xFFFFFFFF);

    int i = 0;
    for (; i + 7 < size; i += 8)
    {
        /* 2. Load 8 elements [v7 v6 v5 v4 v3 v2 v1 v0]: */
        __m256i va_input = _mm256_loadu_si256((__m256i const *)
                            (v.elements));
        /* 3. Split into two sets of 4 : va_input and va_odd */
        // Elements [null, 7, null, 5, null , 3, null, 1]
        __m256i va_odd = _mm256_srli_epi64(va_input, 32);

        /* 4. Process Even Lanes: */
```

```
        __m256i c_even = _shoup_avx(va_input, vb,
                                    vb_precomp, vp, mask_32);

        /* 5. Process Odd Lanes: */
        __m256i c_odd = _shoup_avx(va_odd, vb, vb_precomp,
                                   vp, mask_32);

        /* 6. Shift odd by 32 bits to the left results back
        and merge: */
        // [7, null, 5, null, 3, null, 1, null]
        __m256i res_odd_shifted = _mm256_slli_epi64(c_odd, 32);

        // [7, null, 5, null, 3, null, 1, null]
        //or null, 6, null, 4, null, 2, null, 0]
        __m256i res_combined = _mm256_or_si256(c_even,
                                               res_odd_shifted);
        /* 7. Store: */
        _mm256_storeu_si256((__m256i *)(res.elements),
                            res_combined);
    }
    for (; i < size; i++)
        res.elements[i] = shoup(v.elements[i], param.b,
                                param.b_precomp, param.p);
    return res;
}
```

### Intel _mm256_mullo_epi32

Listing 3: Auxilary Function of Shoup's Modular Reduction (Version mullo)

```
    static inline __m256i _shoup_avx_mullo(__m256i va,
                                           __m256i vb,
                                           __m256i vb_precomp,
                                           __m256i vp)
{
    // 1. q = (a * b_precomp) >> 32:
    __m256i q = _mm256_mul_epu32(va, vb_precomp);
    q = _mm256_srli_epi64(q, 32);

    // 2. ab:
    __m256i ab = _mm256_mullo_epi32(va, vb);

    // 3. qp
    __m256i qp = _mm256_mullo_epi32(q, vp);

    // 4. c = ab - qp
    __m256i c = _mm256_sub_epi32(ab, qp);

    // 5. if (c >= p) c -= p
    __m256i compare = _mm256_cmpgt_epi64(vp, c);
    __m256i sub_mask = _mm256_andnot_si256(compare, vp);

    return _mm256_sub_epi64(c, sub_mask);
}
```

Listing 4: VX2 Implementation of Shoup's Modular Reduction (Version mullo)

```
    static Vector _shoup_scale_avx_mullo(Parameters param, Vector v)
{
    /*1. Preloading b, b_precomp, p and 32 bit mask*/
    int size = v.size;
    Vector res = init_vector(size);
    __m256i vb = _mm256_set1_epi64x(param.b);
    __m256i vb_precomp = _mm256_set1_epi64x(param.b_precomp);
    __m256i vp = _mm256_set1_epi64x(param.p);

    int i = 0;
    for (; i + 31 < size; i += 32)
    {
        /* 2. Load 8 elements [v7 v6 v5 v4 v3 v2 v1 v0]: */
        __m256i va_input_1 = _mm256_loadu_si256((__m256i const *)
                                                 (v.elements));

        /* 3. Split into two sets of 4 : va_input and va_odd */
        // Elements [null, 7, null, 5, null , 3, null, 1]
        __m256i va_odd = _mm256_srli_epi64(va_input, 32);

        /* 4. Process Even Lanes: */
        __m256i c_even = _shoup_avx_mullo(va_input, vb,
                                          vb_precomp, vp);

        /* 5. Process Odd Lanes: */
        __m256i c_odd = _shoup_avx_mullo(va_odd, vb,
                                         vb_precomp, vp);

        /* 6. Shift odd by 32 bits to the left results back
```

```
        and merge: */
        // [7, null, 5, null, 3, null, 1, null]
        __m256i res1_odd_shifted = _mm256_slli_epi64(c_odd, 32);

        // [7, null, 5, null, 3, null, 1, null]
        //or null, 6, null, 4, null, 2, null, 0]
        __m256i res_combined = _mm256_or_si256(c_even_,
                                                res_odd_shifted);

        /* 7. Store: */
        _mm256_storeu_si256((__m256i *)(res.elements + i),
                                        res_combined);
    }
    for (; i < size; i++)
        res.elements[i] = shoup(v.elements[i], param.b,
                                param.b_precomp, param.p);
    return res;
}
```

### 3.8.2 NEON

Listing 5: Auxilary Function of Shoup's Modular Reduction (Version Neon)

```
    TODO
```

Listing 6: NEON Implementation of Shoup's Modular Reduction

```
    TODO
```

## 3.9 Performance

### 3.9.1 Equipement detail:

The performance measurements and benchmarks were performed with the following hardware:

- Processor: AMD Ryzen™ 5 8600G Desktop Processor.
- Architecture: Zen 4 (supporting AVX, AVX2, and AVX-512 instruction sets)
- Compiler : GCC 13.3.0 with optimization flag -O3
- Execution : ./pcca7 -b_bits 31 (Option to set b to 31 bits)

### 3.9.2 Choice of unrolling

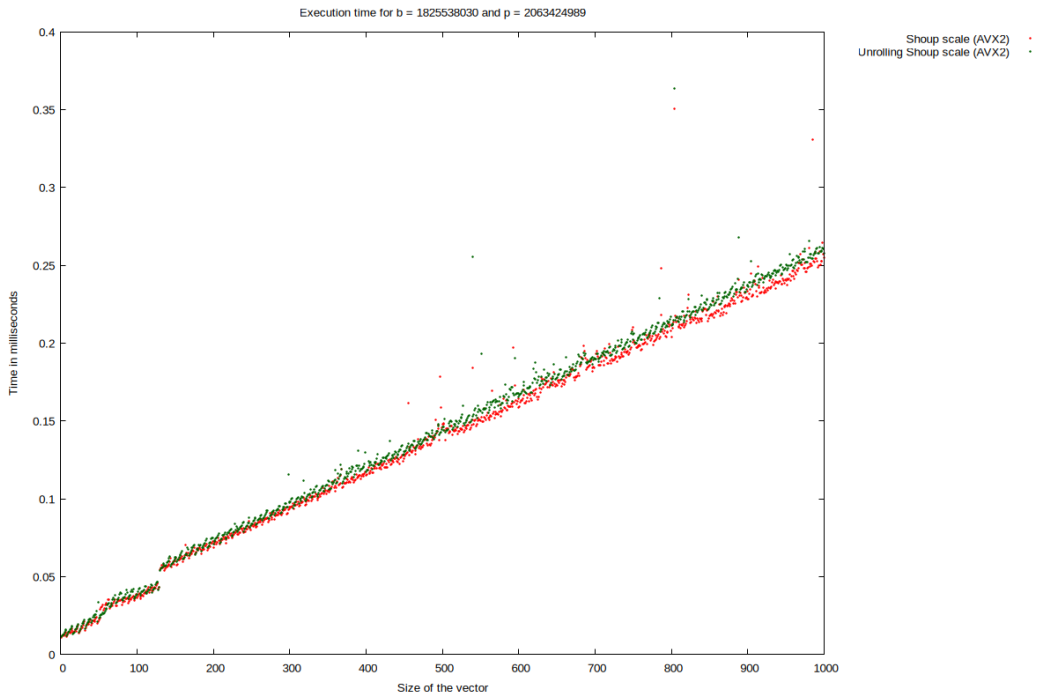


Figure 1: Execution Time Comparison: Standard vs. Unrolled

Table 1: Execution time of implementations with and without unrolling in milliseconds

| Size | Standard | Unrolled |
|---|---|---|
| 100 | 0.037 | 0.039 |
| 200 | 0.069 | 0.071 |
| 300 | 0.093 | 0.098 |
| 400 | 0.115 | 0.118 |
| 500 | 0.148 | 0.146 |
| 600 | 0.159 | 0.166 |
| 700 | 0.195 | 0.190 |
| 800 | 0.204 | 0.212 |
| 900 | 0.228 | 0.237 |
| 1000 | 0.255 | 0.258 |

Table 2: Average percentage speedup of the unrolling implementation relative to with unrolling

| Size | Standard | Unrolled | Percentage difference |
|---|---|---|---|
| 100 | 100% | 105.41% | +5.41% |
| 200 | 100% | 102.90% | +2.90% |
| 300 | 100% | 105.38% | +5.38% |
| 400 | 100% | 102.61% | +2.61% |
| 500 | 100% | 98.65% | −1.35% |
| 600 | 100% | 104.40% | +4.40% |
| 700 | 100% | 97.44% | −2.56% |
| 800 | 100% | 103.92% | +3.92% |
| 900 | 100% | 103.95% | +3.95% |
| 1000 | 100% | 101.18% | +1.18% |

**Obsercation:**On average, the Unrolled implementation is 2.58% slower on average thaan the standard version.

### 3.9.3 Choice between mullo_epi32 and mul_epu32

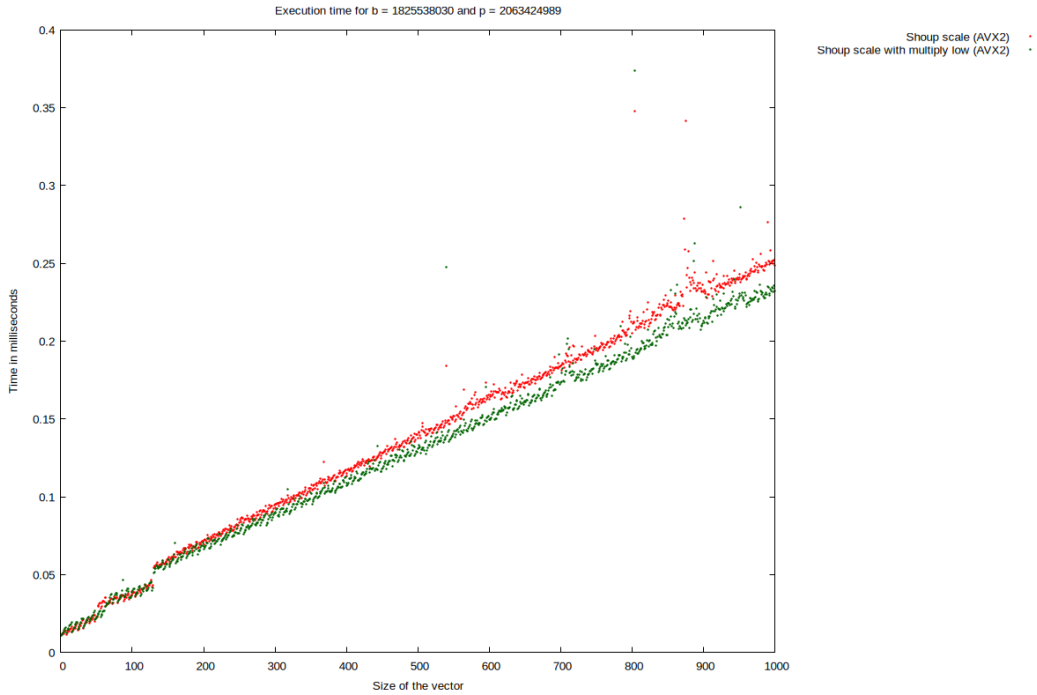


Figure 2: Execution Time Comparison: mullo_epi32 vs. mul_epu32

Table 3: Execution time of mullo_epi32 and mul_epu32 implementation in milliseconds

| Size | mul_epu32 | mullo_epi32 |
|---|---|---|
| 100 | 0.037 | 0.038 |
| 200 | 0.069 | 0.066 |
| 300 | 0.093 | 0.090 |
| 400 | 0.113 | 0.107 |
| 500 | 0.138 | 0.131 |
| 600 | 0.163 | 0.150 |
| 700 | 0.184 | 0.173 |
| 800 | 0.205 | 0.190 |
| 900 | 0.232 | 0.213 |
| 1000 | 0.249 | 0.232 |

Table 4: Average percentage speedup of mullo_epi32 implementation relative to mul_epu32

| Size | mul_epu32 | mullo_epi32 | Percentage difference |
|---|---|---|---|
| 100 | 100% | 102.70% | +2.70% |
| 200 | 100% | 95.65% | −4.35% |
| 300 | 100% | 96.77% | −3.23% |
| 400 | 100% | 94.69% | −5.31% |
| 500 | 100% | 94.93% | −5.07% |
| 600 | 100% | 92.02% | −7.98% |
| 700 | 100% | 94.02% | −5.98% |
| 800 | 100% | 92.68% | −7.32% |
| 900 | 100% | 91.81% | −8.19% |
| 1000 | 100% | 93.17% | −6.83% |

**Observation:** We can observe that at the smaller sizes (100), mullo_epi32 is slower than the standard mul_epu32 implementation by around 2%. But the effecientcy gain starts that show once the vector size hits 200 and above. Across all the vector sizes, the average percentage gain is 5.15%

### 3.9.4 Overall Performance

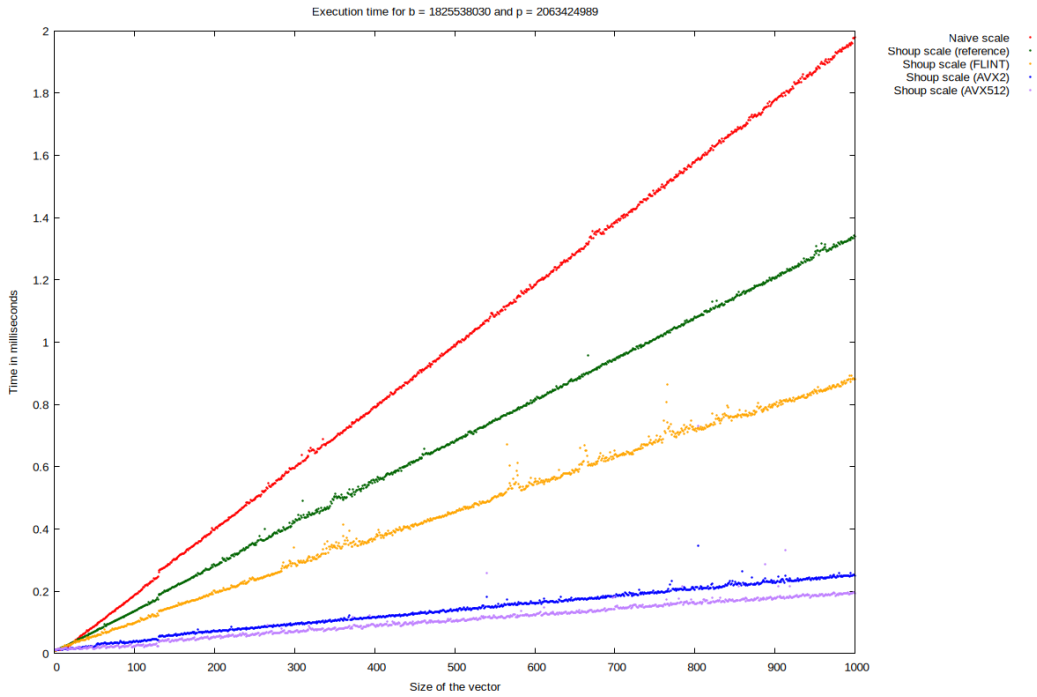


Figure 3: Execution Time Comparison Between Different Implementation For Modular Reduction

Table 5: Execution time for different implementations in relation too the size of the vector in milliseconds

| Size | Naive | Shoup ref | FLINT | Shoup AVX2 | Shoup AVX512 |
|---|---|---|---|---|---|
| 100 | 0.187 | 0.137 | 0.098 | 0.037 | 0.023 |
| 200 | 0.398 | 0.283 | 0.201 | 0.070 | 0.052 |
| 300 | 0.596 | 0.422 | 0.290 | 0.094 | 0.072 |
| 400 | 0.794 | 0.549 | 0.366 | 0.116 | 0.087 |
| 500 | 0.986 | 0.684 | 0.455 | 0.138 | 0.105 |
| 600 | 1.183 | 0.816 | 0.551 | 0.159 | 0.123 |
| 700 | 1.383 | 0.945 | 0.637 | 0.184 | 0.183 |
| 800 | 1.578 | 1.081 | 0.718 | 0.204 | 0.159 |
| 900 | 1.778 | 1.204 | 0.801 | 0.228 | 0.179 |
| 1000 | 1.979 | 1.339 | 0.880 | 0.249 | 0.193 |

Table 6: Average percentage speedup of different algorithms compared to the naive solution

| Size | Naive | Shoup ref | FLINT | Shoup AVX2 | Shoup AVX512 |
|---|---|---|---|---|---|
| 100 | 100% | 73.26% | 52.41% | 19.79% | 12.30% |
| 200 | 100% | 71.11% | 50.50% | 17.59% | 13.07% |
| 300 | 100% | 70.81% | 48.66% | 15.77% | 12.08% |
| 400 | 100% | 69.14% | 46.10% | 14.61% | 10.96% |
| 500 | 100% | 69.37% | 46.15% | 14.00% | 10.65% |
| 600 | 100% | 68.98% | 46.58% | 13.44% | 10.40% |
| 700 | 100% | 68.33% | 46.06% | 13.30% | 13.23% |
| 800 | 100% | 68.50% | 45.50% | 12.93% | 10.08% |
| 900 | 100% | 67.72% | 45.05% | 12.82% | 10.07% |
| 1000 | 100% | 67.66% | 44.47% | 12.58% | 9.75% |

Table 7: Relative Performance Summary in Percentages

| Algorithm | Average % | 1x |
|---|---|---|
| Naive | 100.0% | Baseline |
| Shoup ref | 68.7% | 1.45x |
| FLINT | 45.6% | 2.2x |
| Shoup AVX2 | 15.4% | 6.5x |
| Shoup AVX512 | 11.4% | 8.7x |

**Observation:** From the results retrieved, we can observe that FLINT's implementation over 64 bits provides roughly 2x faster modular reduction on 32 bit operations. Furthermore, our Shoup AVX2 and Shoup AVX512 implementations taillored to 32 bits integers provides significant faster 6-8x speedup.